



NTU Academy for Professional
and Continuing Education

(SCTP) Advanced
Professional Certificate

Data Science and AI





NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

2.10 Event Streaming and Stream Processing

Module Overview

2.1 Introduction to Big Data and Data Engineering

2.2 Data Architecture

2.3 Data Encoding and Data Flow

2.4 Data Extraction and Web Scraping

2.5 Data Warehouse

2.6 Data Pipelines and Orchestration

2.7 Data Orchestration and Testing

2.8 Out of Core/Memory Processing

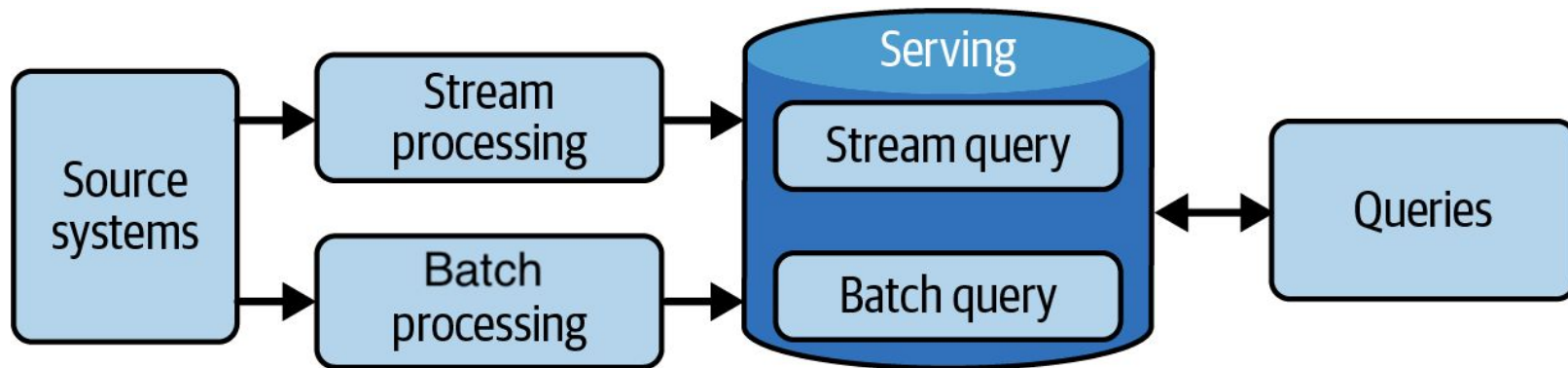
2.9 Big Data Ecosystem and Batch Processing

2.10 Event Streaming and Stream Processing

Agenda

- Event Streaming
- Apache Kafka
- Stream Processing
 - Apache Flink
 - Apache Spark
 - Structured Streaming

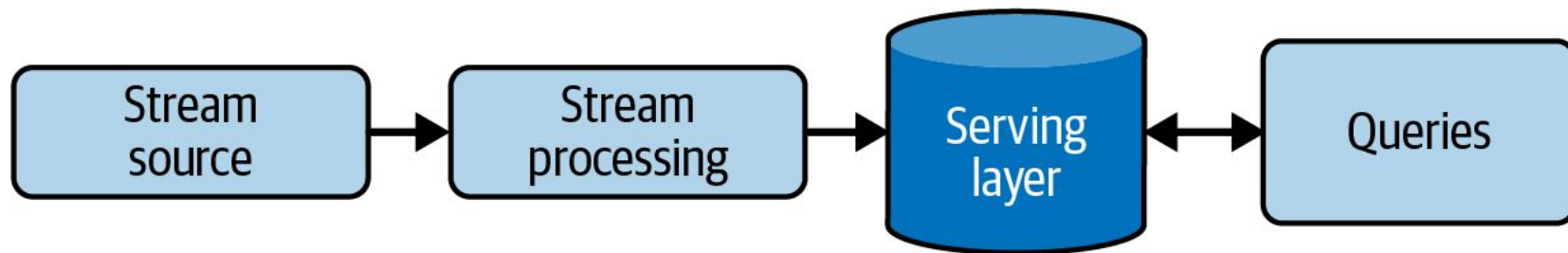
Recap: Lambda Architecture



Recap: Lambda Architecture

- Emerged to handle reconciling batch and streaming data.
 - Separate systems for batch, streaming, and serving data.
 - Source system sends data to both batch and stream destinations.
 - Stream layer focuses on low-latency processing, often using NoSQL.
- Batch layer processes data in systems like data warehouses, creating precomputed/aggregated views.
 - Serving layer aggregates query results from both layers for combined view.
 - Shortcoming: Managing multiple systems, codebases and reconciling code/data.

Recap: Kappa Architecture



Recap: Kappa Architecture

- Proposed as an alternative to Lambda architecture.
 - Suggests using a stream-processing platform for all data tasks - ingestion, storage, serving.
 - Enables true event-based architecture.
- Real-time and batch processing applied to same data by reading live event stream and replaying chunks.
 - **Shortcoming:** Complicated and expensive in practice. Batch storage/processing remains more efficient and cost-effective for enormous historical datasets.



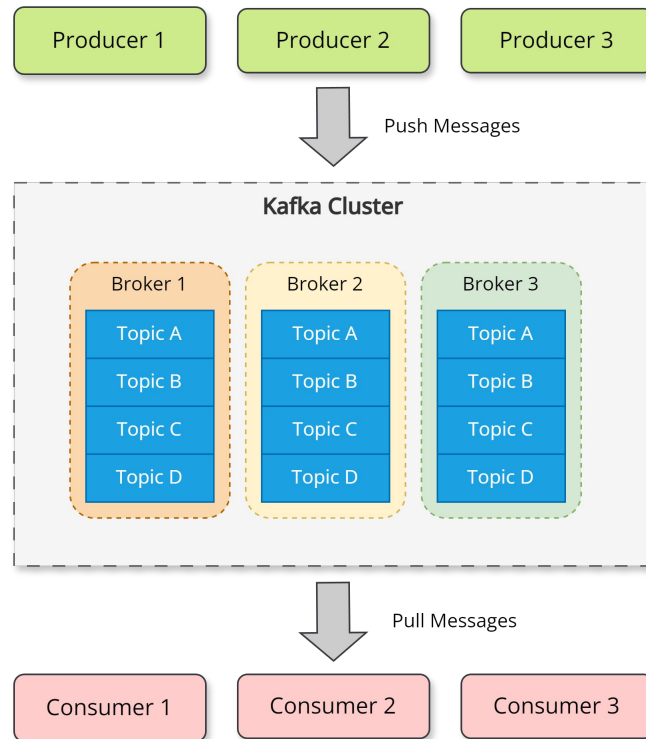
<https://www.youtube.com/watch?v=BsojaA1XnpM>

Apache Kafka Concepts



Topics: A topic is a category or a stream name to which records are published.

Topics in Kafka are always multi-subscriber, i.e a topic can have zero, one, or many consumers that subscribe to the data written to it.



Apache Kafka Concepts

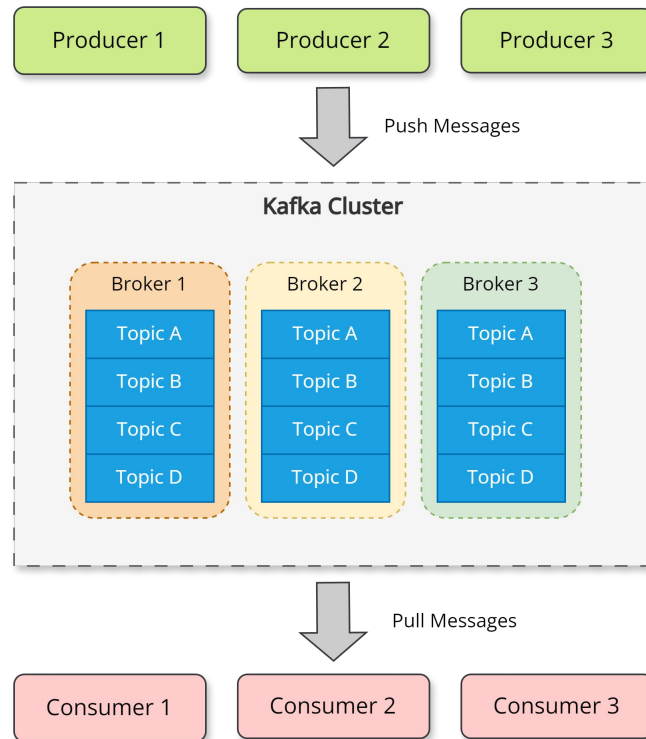


Producers: Applications that publish (or produce) events to Kafka topics.

Consumers: Applications that subscribe to topics and receive the stream of published events.

Brokers: Kafka is cluster-based, and each node in the cluster is called a broker.

A broker receives messages from producers, stores them, and serves them to consumers.



Apache Kafka Concepts

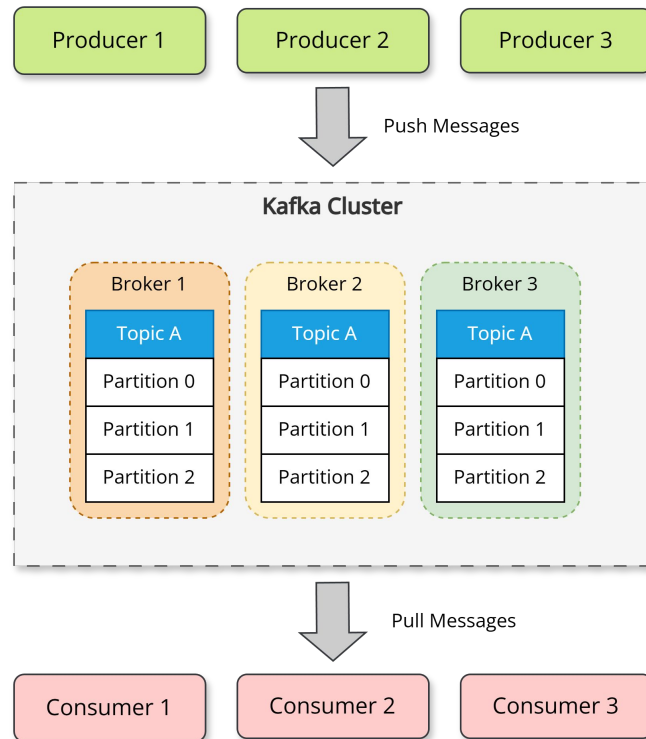


Partitions: Topics can be split into partitions to increase scalability.

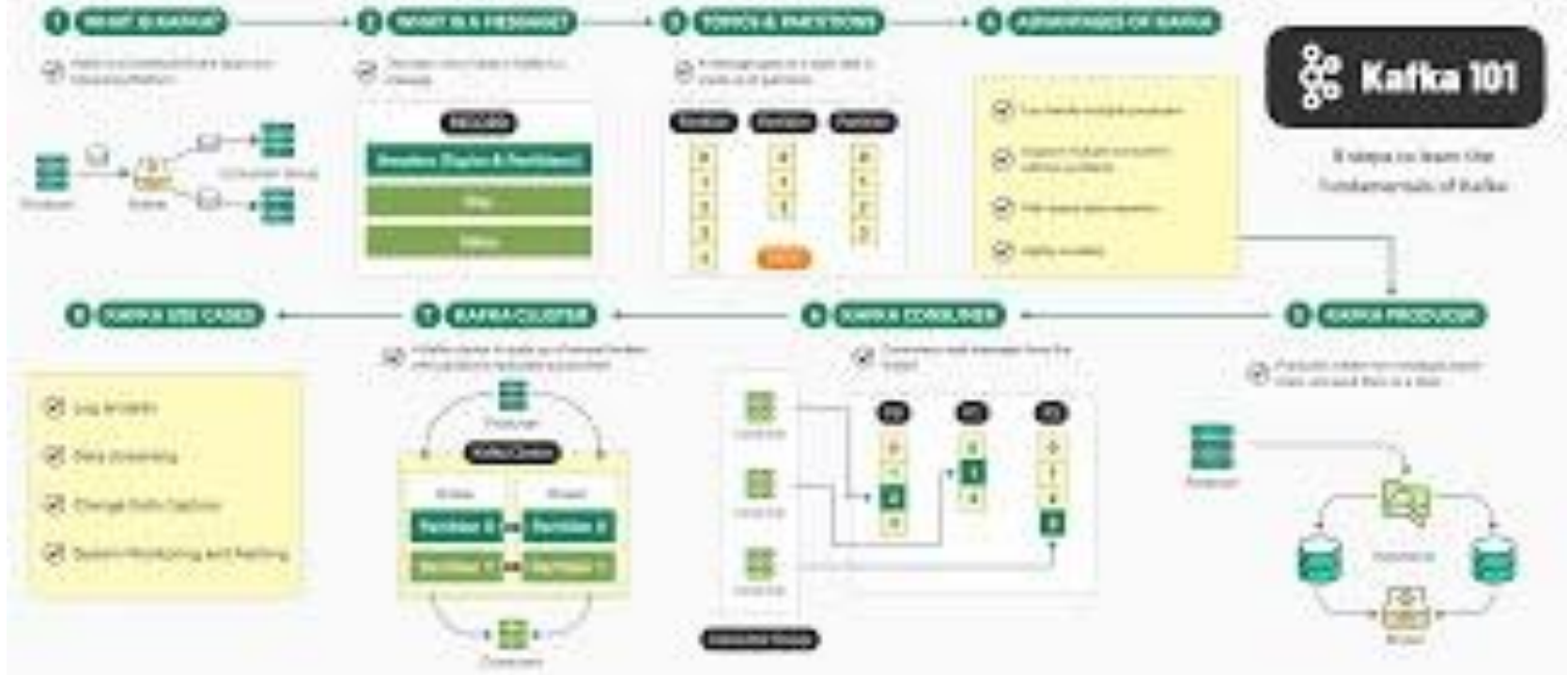
- Each partition can be hosted on a different server, which means that a single topic can be scaled horizontally across multiple brokers.
- Records are appended to a partition in the order they arrive.

Offsets: Within a partition, each record has a unique identifier called an offset.

Consumers use offsets to keep track of messages.



What is Kafka?



<https://www.youtube.com/watch?v=-RDyEFvnTXI>

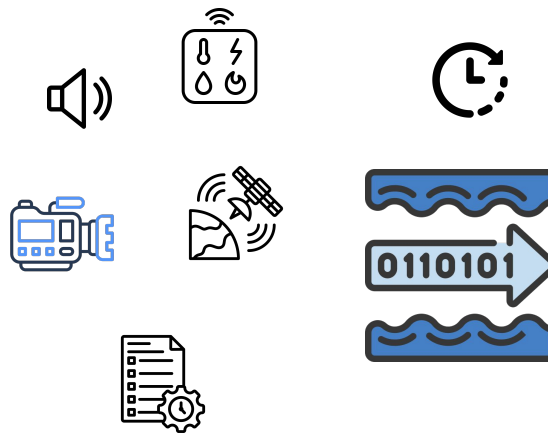
Event Driven System - Class Exercise

- Help a Pizza Company improve order fulfillment with Kafka event driven systems :
 - Determine the processes in Pizza Company order fulfillment
 - Identify the messaging topics
 - Determine the producer / consumer message flow

Stream Processing

Stream processing or *event stream processing* deals with real-time or near-real-time processing of continuous data streams, unlike batch processing of collected data chunks.

A **data stream** is a sequence of data elements, such as log entries, transaction records, or sensor data, made available over time.

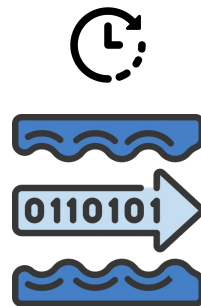


Stream Processing

Each data element in a stream is considered an **event**, which stream processing systems can process, analyze, and act upon in real-time.

To handle continuous streams, data is often bucketed into finite **windows**, either time-based (e.g., last 10 minutes) or event-based (number of events).

Popular stream processing frameworks include Apache Flink and Spark Streaming.

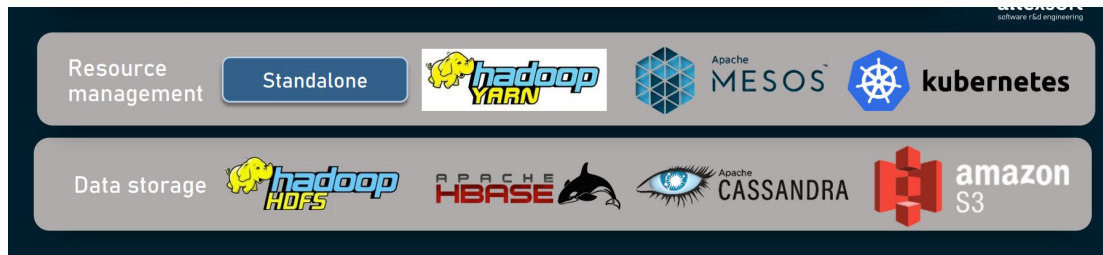
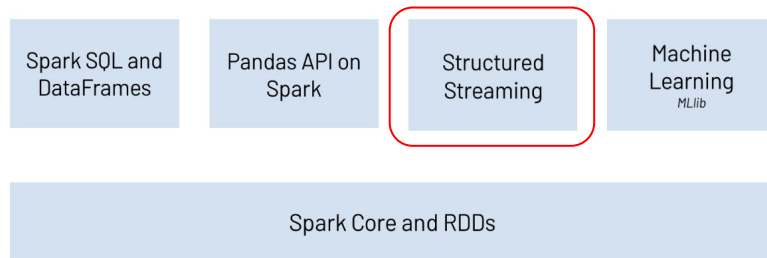


Flink vs Spark Structured Streaming vs Kafka in 60 seconds



<https://www.youtube.com/watch?v=0C5IVDDG2B8>

Apache Spark: Structured Streaming



Structured Streaming is a scalable and fault-tolerant stream processing engine built on the Spark SQL engine. You can express your streaming computation the same way you would express a batch computation on static data.

Stream Processing Fundamentals

Streaming as Tables

Live data streams appear as tables that continuously append new rows.

You write familiar batch-like queries against this unbounded table.

Incremental Processing

The Spark SQL engine runs queries incrementally.

It updates results as new streaming data arrives in real-time.



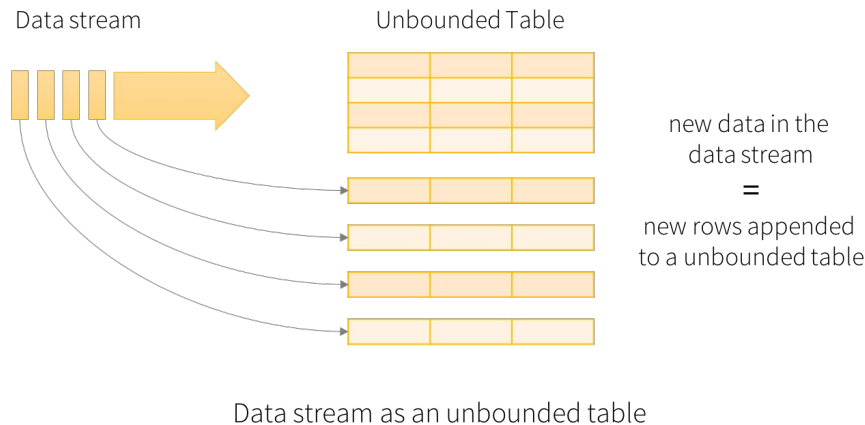
Apache Spark: Structured Streaming



The Spark SQL engine handles incremental and continuous processing, updating results as streaming data arrives.

The **DataFrame** API enables expressing streaming aggregations, event-time windows, stream-to-batch joins, etc., on the Spark SQL engine.

The system ensures end-to-end **exactly-once** fault-tolerance guarantees through *checkpointing* and *Write-Ahead Logs*.

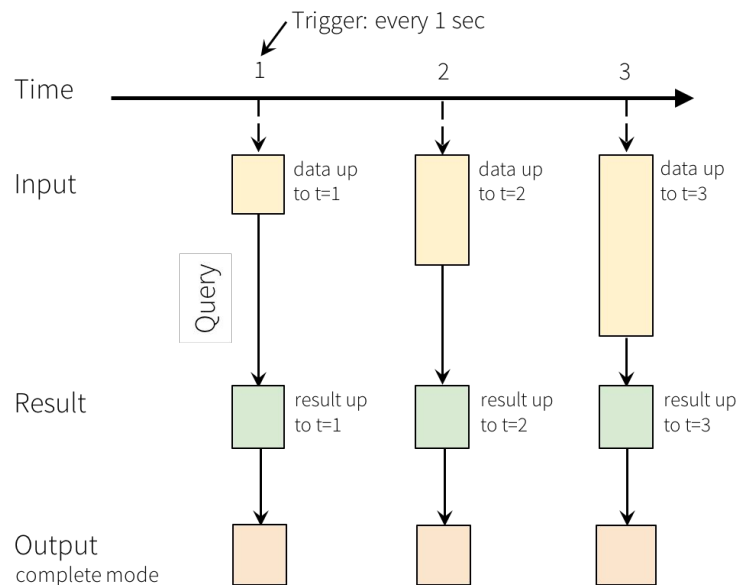


Apache Spark: Structured Streaming



Internally, Structured Streaming uses a **micro-batch** processing engine, achieving end-to-end latencies as low as 100 milliseconds by processing data streams as small batch jobs.

There is also a new processing mode called **Continuous Processing**, which can achieve end-to-end latencies as low as 1 millisecond with at-least-once guarantees.



Programming Model for Structured Streaming

Window Operations and Event Time

Window operations handle time-based aggregations on event time rather than processing time. This ensures accurate analytics with out-of-order data.

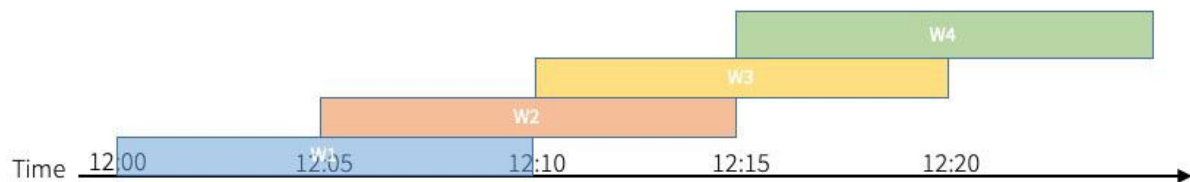
Tumbling Windows (5 mins)



Tumbling Windows

Fixed, non-overlapping time intervals

Sliding Windows (10 mins, slide 5 mins)



Sliding Windows

Overlapping time intervals with slide duration

Session Windows (gap duration 5 mins)



Session Windows

Activity-based windows with timeout for inactivity

Source : <https://www.databricks.com/blog/2021/10/12/native-support-of-session-window-in-spark-structured-streaming.html>

Programming Example

Batch Processing

```
# Batch processing (static data)
df = spark.read.format("json").load("path/to/files")
```

Streaming Processing

```
# Streaming processing (continuous data)
streamingDF =
spark.readStream.format("json").load("path/to/directory")
```

Kafka integration requires minimal configuration for real-time processing:

```
kafkaDF = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()
```